

**LAPORAN IMPLEMENTASI DAN UJI COBA SIMULASI LOAD  
BALANCER NGINX MENGGUNAKAN DOCKER COMPOSE**

*Dosen Pengampuh Mata Kuliah : RUNAL REZKIAWAN S.Kom.,M.T*



**DISUSUN OLEH :**

**NAMA : NURDIAN**

**NIM : 105841118923**

**KELAS : 6A**

**PROGRAM STUDI INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS MUHAMMADIYAH MAKASSAR**

**2026**

## **KATA PENGANTAR**

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan ini dapat diselesaikan dengan baik. Laporan ini disusun untuk memenuhi tugas mata kuliah Sistem Terdistribusi.

Laporan ini membahas implementasi dan pengujian Load Balancer menggunakan NGINX yang dijalankan pada lingkungan Docker Compose. Pada implementasi ini digunakan dua metode load balancing, yaitu Round Robin dan Least Connection, yang masing-masing dijalankan melalui container NGINX terpisah dengan tiga server backend sebagai tujuan distribusi trafik.

Melalui kegiatan ini, penulis memperoleh pemahaman mengenai konsep sistem terdistribusi, khususnya mekanisme pembagian beban trafik pada beberapa server menggunakan teknologi container. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang.

**Makassar, juni 2026**

**Nurdian**

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Perkembangan aplikasi berbasis web menyebabkan kebutuhan akan sistem yang mampu menangani banyak permintaan pengguna secara bersamaan semakin meningkat. Jika seluruh permintaan hanya dilayani oleh satu server, maka server tersebut berpotensi mengalami penurunan performa bahkan kegagalan layanan ketika menerima beban yang tinggi.

Salah satu solusi yang banyak digunakan adalah penerapan **Load Balancer**. Load Balancer berfungsi mendistribusikan permintaan pengguna ke beberapa server backend sehingga beban kerja dapat dibagi secara merata. Dengan demikian, kinerja sistem menjadi lebih stabil dan ketersediaan layanan dapat ditingkatkan.

Pada praktikum ini digunakan **NGINX** sebagai Load Balancer dan **Docker Compose** sebagai platform untuk menjalankan beberapa container secara bersamaan. Pengujian dilakukan menggunakan dua algoritma load balancing, yaitu **Round Robin** dan **Least Connection**, untuk melihat bagaimana masing-masing algoritma mendistribusikan trafik ke tiga server backend.

### **1.2 Rumusan Masalah**

1. Bagaimana cara mengimplementasikan Load Balancer menggunakan NGINX dan Docker Compose?
2. Bagaimana cara mengonfigurasi algoritma Round Robin dan Least Connection pada NGINX?
3. Bagaimana hasil distribusi trafik pada kedua algoritma tersebut?

### **1.3 Tujuan**

1. Mengimplementasikan Load Balancer menggunakan NGINX pada lingkungan Docker.

2. Mengonfigurasi algoritma Round Robin dan Least Connection.
3. Menguji distribusi trafik pada tiga server backend.
4. Membandingkan karakteristik kedua algoritma load balancing.

#### **1.4 Manfaat**

1. Memahami konsep load balancing dalam sistem terdistribusi.
2. Memahami penggunaan Docker Compose untuk membangun layanan multi-container.
3. Mengetahui cara kerja algoritma Round Robin dan Least Connection.
4. Menjadi dasar pembelajaran implementasi infrastruktur server modern.

#### **1.5 Batasan Masalah**

1. Pengujian dilakukan pada lingkungan lokal menggunakan Docker Desktop.
2. Load Balancer yang digunakan adalah NGINX.
3. Backend menggunakan image nginxdemos/hello.
4. Jumlah backend yang digunakan sebanyak tiga container.
5. Pengujian dilakukan menggunakan request HTTP melalui browser dan curl.

## **BAB II**

### **LANDASAN TEORI**

#### **2.1 Load Balancar**

Load Balancer adalah komponen yang bertugas menerima permintaan dari client dan mendistribusikannya ke beberapa server backend. Tujuan utama load balancing adalah meningkatkan performa, ketersediaan layanan, dan efisiensi penggunaan sumber daya server.

#### **2.2 Algoritma Roud Robin**

Round Robin merupakan algoritma load balancing yang mendistribusikan request secara bergantian berdasarkan urutan server yang tersedia. Jika terdapat tiga server backend, maka request akan dibagi secara berurutan ke backend1, backend2, backend3, kemudian kembali ke backend1.

#### **2.3 Algoritma Least Connection**

Least Connection adalah algoritma yang mengarahkan request ke server yang memiliki jumlah koneksi aktif paling sedikit. Metode ini cocok digunakan pada kondisi beban server yang tidak merata karena mampu menyesuaikan distribusi berdasarkan kondisi aktual server.

#### **2.4 Docker Compose**

Docker Compose merupakan alat untuk mendefinisikan dan menjalankan beberapa container Docker melalui satu file konfigurasi YAML. Dengan Docker Compose, seluruh layanan dapat dijalankan menggunakan satu perintah.

#### **2.5 NGINX**

NGINX adalah web server dan reverse proxy yang memiliki performa tinggi. Selain digunakan sebagai web server, NGINX juga banyak digunakan sebagai Load Balancer karena mampu menangani banyak koneksi secara efisien.

## **BAB III**

### **PERANCANGAN DAN IMPLEMENTASI SISTEM**

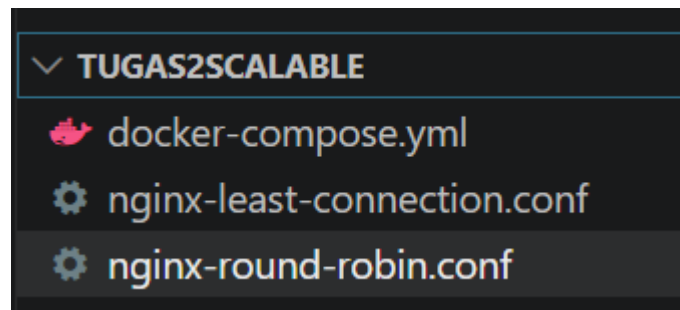
#### **3.1 Arsitektur Sistem**

Sistem terdiri dari:

- 1 Load Balancer NGINX Round Robin (Port 8081)
- 1 Load Balancer NGINX Least Connection (Port 8082)
- 3 Server Backend:
  - backend1
  - backend2
  - backend3

Seluruh container berjalan dalam jaringan Docker yang sama sehingga dapat saling berkomunikasi menggunakan nama service.

#### **3.2 Struktur Folder**



#### **3.3 Implementasi Docker Compose**

```
version: '3.8'

services:
  lb-roundrobin:
    image: nginx:latest
    container_name: simulasi-lb-roundrobin
    volumes:
      - ./nginx-round-robin.conf:/etc/nginx/nginx.conf:ro
    ports:
      - "8081:80"
    depends_on:
      - backend1
      - backend2
      - backend3

  lb-leastconn:
    image: nginx:latest
    container_name: simulasi-lb-leastconn
    volumes:
      - ./nginx-least-connection.conf:/etc/nginx/nginx.conf:ro
    ports:
      - "8082:80"
    depends_on:
      - backend1
      - backend2
      - backend3

  backend1:
    image: nginxdemos/hello
    container_name: simulasi-lb-backend1

  backend2:
    image: nginxdemos/hello
    container_name: simulasi-lb-backend2

  backend3:
    image: nginxdemos/hello
    container_name: simulasi-lb-backend3
```

### 3.4 Konfigurasi Round Robin

```
nginx-round-robin.conf
1  events {}
2
3  http {
4      upstream backend_servers {
5          server backend1:80;
6          server backend2:80;
7          server backend3:80;
8      }
9
10     server {
11         listen 80;
12
13         location / {
14             proxy_pass http://backend_servers;
15         }
16     }
17 }
```

### 3.5 Konfigurasi Least Connection

```
1  events {}
2
3  http {
4      upstream backend_servers {
5          least_conn;
6
7          server backend1:80;
8          server backend2:80;
9          server backend3:80;
10     }
11
12     server {
13         listen 80;
14
15         location / {
16             proxy_pass http://backend_servers;
17         }
18     }
19 }
```

### 3.6 Menjalankan Sistem

- **Menjalankan Seluruh Container**

```
PS C:\Users\ASUS\Downloads\tugas2scalable> docker-compose up -d
```

- **Memastikan Container Aktif**

```
PS C:\Users\ASUS\Downloads\tugas2scalable> docker ps
```

- **Mengakses Load Balancer Round Robin**

```
PS C:\Users\ASUS\Downloads\tugas2scalable> start http://localhost:8081
```

- **Mengakses Load Balancer Least Connection**

```
PS C:\Users\ASUS\Downloads\tugas2scalable> start http://localhost:8082
```



## **BAB IV**

### **HASIL DAN PEMBAHASAN**

#### **4.1 Skenario Penulisan**

Pengujian dilakukan setelah seluruh container berhasil dijalankan menggunakan Docker Compose.

#### **4.2 Hasil Pengujian Pengujian Least Connection**

Saat mengakses port 8082, request didistribusikan ke backend yang memiliki jumlah koneksi aktif paling sedikit.

Karena pengujian dilakukan secara sederhana dan setiap request selesai dengan cepat, distribusi yang terlihat cenderung merata di antara backend1, backend2, dan backend3.

#### **4.3 Analisis Hasil Least Connection**

Pada kondisi trafik ringan, seluruh backend hampir selalu memiliki jumlah koneksi aktif yang sama. Oleh karena itu, pola distribusi terlihat menyerupai Round Robin. Perbedaan algoritma akan lebih terlihat ketika terdapat banyak koneksi aktif secara bersamaan.

#### **4.4 Hasil Pengujian Round Robin**

Saat mengakses port 8081, request didistribusikan secara bergiliran ke backend1, backend2, dan backend3.

Pola distribusi yang dihasilkan bersifat tetap dan berulang.

#### **4.5 Analisis Hasil Round Robin**

Round Robin membagi request tanpa mempertimbangkan kondisi server. Setiap backend memperoleh jumlah request yang relatif sama sehingga cocok digunakan pada lingkungan server yang memiliki spesifikasi seragam.

#### **4.6 Perbandingan Round Robin dan least Connection**

| <b>Aspek</b> | <b>Round Robin</b> | <b>Least Connection</b> |
|--------------|--------------------|-------------------------|
|              |                    |                         |

|              |              |                                  |
|--------------|--------------|----------------------------------|
| Cara kerja   | Bergiliran   | Berdasarkan Jumlah Koneksi aktif |
| Distribusi   | Statis       | Dinamis                          |
| Kompleksitas | Sederhana    | Lebih adaptif                    |
| Cocok Untuk  | Beban Merata | Beban tidak merata               |
| Efesiensi    | Baik         | Lebih baik pada trafik tinggi    |

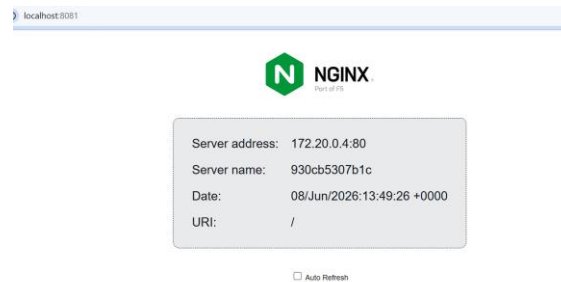
## 4.7 Bukti Pengujian

### a. Hasil docker ps

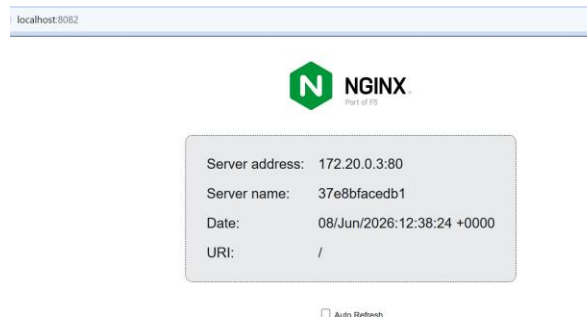
```
PS C:\Users\VGUS\Downloads\luginscalabzo> docker ps
```

| CONTAINER ID | IMAGE           | COMMAND                 | CREATED        | STATUS        | PORTS                                   | NAMES                  |
|--------------|-----------------|-------------------------|----------------|---------------|-----------------------------------------|------------------------|
| a6d7c3b600b  | nginx:latest    | "/docker-entrypoint..." | 16 seconds ago | Up 13 seconds | 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp | simulasi-lb-roundrobin |
| 26d56f43352  | nginx:latest    | "/docker-entrypoint..." | 16 seconds ago | Up 13 seconds | 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp | simulasi-lb-leastconn  |
| 89d8e55866   | nginxdemo:hello | "/docker-entrypoint..." | 16 seconds ago | Up 14 seconds | 80/tcp                                  | simulasi-lb-backend1   |
| 37e8bfacedb1 | nginxdemo:hello | "/docker-entrypoint..." | 16 seconds ago | Up 14 seconds | 80/tcp                                  | simulasi-lb-backend2   |
| 930cb53071c  | nginxdemo:hello | "/docker-entrypoint..." | 16 seconds ago | Up 14 seconds | 80/tcp                                  | simulasi-lb-backend3   |

### b. Hasil akses <http://localhost:8081>



### c. Hasil akses <http://localhost:8082>



## **BAB V**

### **PENUTUP**

#### **5.1 Kesimpulan**

- 1. Load Balancer berhasil diimplementasikan menggunakan NGINX dan Docker Compose.**
- 2. Sistem terdiri dari dua Load Balancer dan tiga server backend.**
- 3. Algoritma Round Robin berhasil mendistribusikan request secara bergiliran.**
- 4. Algoritma Least Connection berhasil memilih backend berdasarkan jumlah koneksi aktif.**
- 5. Pada kondisi pengujian sederhana, kedua algoritma menghasilkan distribusi yang relatif merata.**

#### **5.2 Saran**

- 1. Melakukan pengujian menggunakan tool load testing seperti Apache JMeter atau Locust.**
- 2. Menambahkan lebih banyak backend untuk menguji skalabilitas sistem.**
- 3. Menggunakan monitoring seperti Grafana dan Prometheus untuk mengamati performa container secara real-time.**

## DAFTAR PUSTAKA

- Buyya, R., Broberg, J., & Goscinski, A. (2019). *Cloud Computing: Principles and Paradigms*. Wiley.
- Docker Inc. (2026). *Docker Compose Documentation*. Tersedia pada: <https://docs.docker.com/compose/>
- Khan, A. M., Othman, M., Madani, S. A., & Khan, S. U. (2014). *A Survey of Mobile Cloud Computing Application Models*. IEEE Communications Surveys & Tutorials.
- Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing*. National Institute of Standards and Technology (NIST).
- NGINX Inc. (2026). *NGINX Load Balancing Documentation*. Tersedia pada: [https://nginx.org/en/docs/http/load\\_balancing.html](https://nginx.org/en/docs/http/load_balancing.html)
- Randles, M., Lamb, D., & Taleb-Bendiab, A. (2010). *A Comparative Study into Distributed Load Balancing Algorithms for Cloud Computing*. IEEE International Conference on Advanced Information Networking and Applications Workshops.
- Stallings, W. (2019). *Data and Computer Communications* (10th Edition). Pearson Education.